MODUL 04 — Membangun Aplikasi Web MVC: Sistem Inventaris Barang (PHP & MySQL)

Mata Kuliah: Pemrograman Web (PHP dan MySQL) • **Modul ke-:** 4 **Alokasi Waktu:** 3 × 50 menit (teori + praktikum) dan dapat dilanjutkan sebagai proyek mandiri **Prasyarat:** Modul 02 (Dasar PHP), Modul 03 (Struktur Kontrol), dasar HTML/CSS, dasar SQL/MySQL, dasar Git **Bahasa Implementasi:** PHP 8.x + MySQL/MariaDB (tanpa framework, arsitektur MVC murni)

Capaian Pembelajaran

Setelah mengikuti modul ini, mahasiswa mampu **membangun aplikasi web dinamis dari nol** menggunakan pola **Model-View-Controller (MVC)** tanpa framework: membuat *front controller* dan routing, memisahkan tanggung jawab Model–View–Controller, mengakses MySQL dengan **PDO prepared statement**, menerapkan **CRUD**, mengelola **autentikasi berbasis session**, mencatat **transaksi stok** secara aman, serta menyusun tampilan dengan **paginasi dan pencarian**.

Konvensi Kode Modul Ini

Seluruh kode memakai PHP 8.x, PDO, dan gaya PSR-12. Aplikasi dijalankan dengan PHP built-in server maupun XAMPP. Semua kode lengkap tersedia sebagai referensi pada folder `../inventory-mvc`; modul ini menuntun Anda **membangunnya sendiri langkah demi langkah**.

Strategi Belajar (PENTING)

Modul ini adalah **modul latihan bertahap**. Jangan langsung menyalin folder `../inventory-mvc`. Bangun aplikasi **satu langkah setiap kali**, lalu uji pada *checkpoint* di akhir tiap langkah. Setiap langkah dirancang agar aplikasi tetap dapat dijalankan meskipun fitur belum lengkap.

Daftar Isi

- **BAGIAN 1: PENDAHULUAN MODUL**
  1. Deskripsi Singkat Modul
  2. Capaian Pembelajaran (Sub-CPMK)
  3. Petunjuk Penggunaan Modul
  4. Peta Konsep / Mind Map
- **BAGIAN 2: KEGIATAN BELAJAR (PRAKTIK BERTAHAP)**
  1. Gambaran Arsitektur & Alur Request
  2. Langkah 0 — Persiapan Lingkungan
  3. Langkah 1 — Struktur Folder & Konfigurasi `.env`
  4. Langkah 2 — Front Controller & Autoloader
  5. Langkah 3 — Router & Base Controller (Hello MVC)
  6. Langkah 4 — Koneksi Database (PDO) & Base Model
  7. Langkah 5 — Skema Database & Data Contoh
  8. Langkah 6 — Layout View & Helper
  9. Langkah 7 — CRUD Kategori (Model → Controller → View)
  10. Langkah 8 — CRUD Barang
  11. Langkah 9 — Transaksi Stok Masuk/Keluar
  12. Langkah 10 — Autentikasi & Proteksi Route
  13. Langkah 11 — Paginasi, Pencarian, & Filter
  14. Langkah 12 — Dashboard Dinamis
  15. Langkah 13 — Halaman 404, Logging, & Perapian Helper
  16. Langkah 14 — Pengujian Akhir & Checklist Serah Terima
  17. Isu Terkini & Praktik Industri
- **BAGIAN 3: RANGKUMAN & EVALUASI PEMBELAJARAN**
  1. Rangkuman
  2. Latihan Soal & Tugas Mandiri
- **BAGIAN 4: DAFTAR PUSTAKA & GLOSARIUM**
  1. Glosarium
  2. Daftar Pustaka

BAGIAN 1: PENDAHULUAN MODUL

1. Deskripsi Singkat Modul

Pada modul-modul sebelumnya, mahasiswa telah menguasai "bahan baku" (variabel, operator), "kendali alur" (percabangan, perulangan), dasar fungsi/array/form, SQL, serta PDO untuk CRUD. Namun, kode yang ditulis secara **prosedural** (satu berkas mencampur HTML, query, dan logika) cepat menjadi sulit dirawat ketika aplikasi membesar. Modul ini menjawab masalah tersebut dengan **arsitektur Model-View-Controller (MVC)**.

MVC memisahkan tiga tanggung jawab: **Model** (data & aturan akses database), **View** (tampilan/template), dan **Controller** (pengendali alur: menerima request, memanggil Model, mengirim data ke View). Sebuah **front controller** (`public/index.php`) menjadi satu-satunya pintu masuk, dan sebuah **router** memetakan URL ke Controller. Pemahaman MVC adalah jembatan menuju framework modern (Laravel, CodeIgniter, Symfony) yang memakai pola sama.

Sebagai studi kasus, kita membangun **Sistem Inventaris Barang**: mengelola kategori dan barang, mencatat **barang masuk/keluar** (transaksi stok) dengan menjaga stok agar tidak negatif, serta **login** untuk membatasi akses. Aplikasi dibangun **tanpa framework** agar setiap mekanisme — routing, autoload, PDO, session, middleware sederhana — terlihat jelas dan dapat dipahami.

2. Capaian Pembelajaran (Sub-CPMK)

Setelah mempelajari modul ini secara utuh dan mengerjakan seluruh langkah serta tugas, mahasiswa diharapkan mampu:

1. **Menjelaskan** prinsip *separation of concerns* dan tanggung jawab Model, View, Controller. *(CPMK4)*
2. **Membangun** kerangka MVC sederhana tanpa framework mencakup front controller, autoloader, dan router. *(CPMK4)* **[Indikator utama]**
3. **Mengimplementasikan** koneksi **PDO** dan *prepared statement* di dalam layer Model. *(CPMK2, CPMK4)*
4. **Membuat** operasi **CRUD** lengkap (create, read, update, delete) yang terpisah rapi antar-layer. *(CPMK2, CPMK4)*
5. **Menerapkan session** dan mekanisme **middleware** sederhana untuk autentikasi dan proteksi route. *(CPMK5)*
6. **Menerapkan database transaction** untuk operasi yang mengubah beberapa tabel sekaligus (stok + riwayat transaksi). *(CPMK5)*
7. **Menyusun** daftar data dengan **paginasi, pencarian, dan filter** yang aman. *(CPMK2)*
8. **Mengelola** konfigurasi aplikasi dengan berkas `.env` dan menerapkan keamanan dasar (*escape* output, hashing password). *(CMK5)*
9. **Menguji**, mendokumentasikan, dan mengelola proyek dengan **Git**. *(CPMK3, CPMK5)*

> **Indikator Ketercapaian (Sub-CPMK):** *Pemisahan layer jelas, routing berjalan, CRUD dan transaksi berfungsi, serta autentikasi dan pembatasan akses bekerja.*

3. Petunjuk Penggunaan Modul

A. Alur Belajar Mahasiswa

Tahap	Aktivitas	Keluaran
1	Membaca **Bagian 1** (tujuan, peta konsep).	Gambaran besar MVC
2	Mengerjakan **Langkah 0–3** (environment, config, routing).	Aplikasi "Hello MVC" berjalan
3	Mengerjakan **Langkah 4–5** (PDO, Model, skema DB).	Koneksi DB & struktur tabel
4	Mengerjakan **Langkah 6–8** (layout, Kategori, Barang).	CRUD master data berjalan
5	Mengerjakan **Langkah 9–10** (transaksi stok, autentikasi).	Stok masuk/keluar + login
6	Mengerjakan **Langkah 11–13** (paginasi, dashboard, 404/log).	Aplikasi utuh & rapi
7	Menyelesaikan **Langkah 14, latihan, dan tugas**.	Proyek siap kumpul + refleksi

B. Persiapan Sebelum Memulai

- Sudah menguasai Modul 02–03 dan CRUD PDO (minggu 6–7).
- Menyiapkan **Git** dan akun repositori (GitHub/GitLab) untuk mengumpulkan proyek.
- Mampu menjalankan perintah dasar terminal.

C. Etika Belajar

- **Ketik ulang** kode, jangan *copy-paste* dari `../inventory-mvc` tanpa memahami.
- Uji setiap **checkpoint** sebelum lanjut; bug yang menumpuk sulit dilacak.
- Biasakan menulis **pesan commit** yang jelas setiap menyelesaikan satu langkah.

D. Konvensi Perintah (Windows & Linux/macOS)

Modul ini banyak memakai perintah terminal. Sebagian besar perintah (`php`, `git`, `mysql`) sama di semua sistem, tetapi beberapa berbeda. Gunakan tabel padanan berikut:

Perintah di modul (Linux/macOS)	Windows — Command Prompt	Windows — PowerShell
`php -v`	`php -v`	`php -v`
`php -m | grep -i pdo_mysql`	`php -m | findstr /i pdo_mysql`	`php -m | Select-String pdo_mysql`
`ls`	`dir`	`Get-ChildItem`
`mkdir -p a b c`	`mkdir a b c` (bila gagal, buat satu per satu)	`New-Item -ItemType Directory a, b, c`
`rm -rf folder`	`rmdir /s /q folder`	`Remove-Item -Recurse -Force folder`
`mysql -u root -p < file.sql`	`mysql -u root -p < file.sql`	`Get-Content file.sql | mysql -u root -p`
`cmd1 && cmd2`	`cmd1 && cmd2`	`cmd1; cmd2`

Tips Windows

- Bila `php` belum dikenali, pakai path lengkap XAMPP: `C:\xampp\php\php.exe`; untuk MySQL: `C:\xampp\mysql\bin\mysql.exe`. Atau tambahkan `C:\xampp\php` ke variabel **PATH**.
- Redirection `<` **tidak** bekerja di PowerShell — gunakan `Get-Content ... |`.
- Untuk perintah `php -r "..."`, gunakan **tanda kutip ganda** seperti contoh pada modul.
- Pemisah folder di Windows memakai `\` (mis. `public\index.php`), sedangkan di `.env`/URL tetap `/`.

4. Peta Konsep / Mind Map

```
APLIKASI MVC — SISTEM INVENTARIS
│
├── A. PINTU MASUK (Front Controller)
│   ├── public/index.php       (bootstrap: config, autoload, session, router)
│   └── .htaccess              (rewrite URL -> index.php?url=...)
│
├── B. INTI (core/)
│   ├── App.php                (router: /controller/method/param)
│   ├── Controller.php         (view(), model(), redirect(), flash(), requireLogin())
│   ├── Database.php           (PDO singleton)
│   ├── Model.php              (query dasar: all, find, delete, count)
│   ├── Middleware.php         (isLoggedIn, requireLogin)
│   ├── Logger.php             (catat error ke berkas)
│   └── helpers.php            (e(), rupiah(), base_url(), take_flash())
│
├── C. MODEL (app/models/)
│   ├── Category.php           (kategori)
│   ├── Item.php               (barang + paginasi + stok menipis)
│   ├── Transaction.php        (stok masuk/keluar + DB transaction)
│   └── User.php               (login + daftar pengguna)
│
├── D. VIEW (app/views/)
│   ├── layouts/               (header, footer)
```

```
│   ├── dashboard/  items/  categories/  transactions/  users/
│   ├── auth/login.php
│   └── errors/404.php
│
├── E. CONTROLLER (app/controllers/)
│   ├── HomeController / DashboardController
│   ├── ItemsController / CategoriesController
│   ├── TransactionsController
│   ├── UsersController
│   └── AuthController
│
└── F. DATA & KONFIGURASI
    ├── .env                    (BASE_URL, kredensial DB)
    └── database/inventory.sql  (users, categories, items, transactions)
```

BAGIAN 2: KEGIATAN BELAJAR (PRAKTIK BERTAHAP)

1. Gambaran Arsitektur & Alur Request

Perhatikan alur satu buah *request* dari browser sampai menghasilkan halaman (contoh: membuka `/items`):

```
Browser  ->  GET /items
   |
   ▼
public/.htaccess        rewrites  /items  ->  index.php?url=items
   |
   ▼
public/index.php        bootstrap: config + helpers + autoloader + session
   |
   ▼
core/App.php            pecah url -> controller=Items, method=index
   |
   ▼
ItemsController         requireLogin() -> model('Item')->paginate(...)
   |          |
   |          └──>  Item.php  --PDO-->  MySQL (db_inventory)
   ▼
Controller::view()      extract(data) -> render views/items/index.php
   |
   ▼
views (header+content+footer)  ->  HTML  ->  Browser
```

Struktur folder akhir yang akan kita bangun:

```
inventaris-mvc/
├── .env
├── .env.example
├── .gitignore
├── app/
│   ├── config/config.php
│   ├── core/  (App, Controller, Database, Model, Middleware, Logger, helpers)
│   ├── controllers/  (Home, Dashboard, Items, Categories, Transactions, Users, Auth)
│   ├── models/  (Category, Item, Transaction, User)
│   └── views/  (layouts, dashboard, items, categories, transactions, users, auth, errors)
├── database/inventory.sql
├── public/
│   ├── .htaccess
│   └── index.php
└── .htaccess
```

> **Tips Git:** inisialisasi repositori **sekarang** (`git init`) dan lakukan commit pertama setelah Langkah 1. Commit tiap langkah, mis. `feat: langkah 3 - router dan base controller`.

Langkah 0 — Persiapan Lingkungan

Tujuan: memastikan semua *tools* siap dan aplikasi PHP sederhana dapat dijalankan.

Praktik

1. Pastikan PHP 8.x dan ekstensi PDO MySQL terpasang:

```
php -v
php -m | grep -i pdo_mysql       # Windows: php -m | findstr /i pdo_mysql
```

2. Pastikan MySQL/MariaDB berjalan dan Anda dapat masuk:

```
mysql -u root -p -e "SELECT VERSION();"
```

3. Buat folder proyek dan masuk ke dalamnya:

```
mkdir inventaris-mvc && cd inventaris-mvc   # Windows: mkdir inventaris-mvc  lalu  cd inventaris-mvc
git init
```

Pengguna Windows (XAMPP): semua perintah pada modul ini dapat dijalankan di Windows. Isi baris `2` dan `3` di bawah digantikan padanannya:

Langkah	Linux/macOS	Windows (Command Prompt / PowerShell)
Cek PHP	`php -v`	`C:\xampp\php\php.exe -v`
Cek ekstensi	`php -m | grep -i pdo_mysql`	`C:\xampp\php\php.exe -m | findstr /i pdo_mysql`
Buat folder	`mkdir inventaris-mvc && cd inventaris-mvc`	`mkdir inventaris-mvc` lalu `cd inventaris-mvc`

Penting: XAMPP **sudah** menyediakan `php`, `mysql`, dan Apache. Anda bisa memakainya langsung dari `C:\xampp\php\` dan `C:\xampp\mysql\bin\`. Letakkan proyek di `C:\xampp\htdocs`, akses lewat `http://localhost/<nama-folder>/public`, dan impor `database/inventory.sql` melalui phpMyAdmin (`http://localhost/phpmyadmin`).

Checkpoint: `php -v` menampilkan PHP ≥ 8.0, `pdo_mysql` muncul, dan MySQL dapat diakses.

Kesalahan umum: ekstensi `pdo_mysql` tidak aktif → aktifkan pada `php.ini` (`extension=pdo_mysql`).

Langkah 1 — Struktur Folder & Konfigurasi `.env`

Tujuan: membuat kerangka folder dan memusatkan konfigurasi (base URL, database) pada berkas `.env`.

Konsep: jangan pernah menaruh kredensial langsung di kode. Gunakan berkas `.env` yang **tidak** di-commit, dan sediakan `.env.example` sebagai templat.

Praktik

1. Buat struktur folder:

```
mkdir -p app/config app/core app/controllers app/models \
        app/views/layouts app/views/dashboard app/views/items \
        app/views/categories app/views/transactions app/views/users \
        app/views/auth app/views/errors public database
```

> **Windows (PowerShell):** `New-Item -ItemType Directory app\config, app\core, app\controllers, app\models, app\views\layouts, app\views\dashboard, app\views\items, app\views\categories, app\views\transactions, app\views\users, app\views\auth, app\views\errors, public, database`

2. Buat `.env`:

```
APP_NAME="Inventory System"
APP_ENV=local
APP_DEBUG=true
APP_TIMEZONE=Asia/Jakarta

# Base URL. Sesuaikan dengan cara aplikasi diakses.
#   PHP built-in server : http://localhost:8000
#   XAMPP (via public)  : http://localhost/inventory-mvc/public
#   Virtual host        : http://inventory-mvc.test
BASE_URL=http://localhost:8000

DB_HOST=localhost
DB_PORT=3306
DB_NAME=db_inventory
DB_USER=root
DB_PASS=123456
```

3. Salin menjadi `.env.example` (kosongkan `DB_PASS`) dan buat `.gitignore`:

```
.env
logs/
```

4. Buat `app/config/config.php` dengan *loader* `.env` + helper `env()`:

```php
<?php
define('ROOT_PATH', dirname(__DIR__, 2));

function loadEnv(string $file): void {
    if (!file_exists($file)) return;
    foreach (file($file, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES) as $line) {
        $line = trim($line);
        if ($line === '' || str_starts_with($line, '#') || !str_contains($line, '=')) continue;
        [$name, $value] = array_map('trim', explode('=', $line, 2));
        $_ENV[$name] = trim($value, "\"'");
        putenv("$name=$value");
    }
}
loadEnv(ROOT_PATH . '/.env');

function env(string $key, $default = null) {
    $value = $_ENV[$key] ?? getenv($key);
    return ($value === false || $value === null || $value === '') ? $default : $value;
}

define('APP_NAME', env('APP_NAME', 'Inventory System'));
define('APP_ENV', env('APP_ENV', 'production'));
define('APP_DEBUG', filter_var(env('APP_DEBUG', false), FILTER_VALIDATE_BOOLEAN));
```

```
define('BASEURL', rtrim(env('BASE_URL', 'http://localhost:8000'), '/'));
date_default_timezone_set(env('APP_TIMEZONE', 'Asia/Jakarta'));

define('DB_HOST', env('DB_HOST', 'localhost'));
define('DB_PORT', env('DB_PORT', '3306'));
define('DB_USER', env('DB_USER', 'root'));
define('DB_PASS', env('DB_PASS', ''));
define('DB_NAME', env('DB_NAME', 'db_inventory'));
```

5. Commit pertama:

```
git add . && git commit -m "chore: struktur proyek dan konfigurasi env"
```

Checkpoint:

```
php -r "require 'app/config/config.php'; echo BASE_URL ?? ''; echo BASEURL, PHP_EOL;"
```

Menampilkan `http://localhost:8000`.

Kesalahan umum: lupa `rtrim()` pada `BASEURL` sehingga muncul `//` pada URL; `dirname(__DIR__, 2)` salah jumlah level.

Langkah 2 — Front Controller & Autoloader

Tujuan: membuat satu pintu masuk `public/index.php` yang memuat konfigurasi, *autoloader*, dan session.

Konsep: Front controller adalah satu-satunya berkas yang diakses langsung. **Autoloader** (`spl_autoload_register`) memuat class sesuai nama sehingga tidak perlu `require` manual.

Praktik

1. Buat `public/.htaccess`:

```
Options -Multiviews
<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteCond %{REQUEST_FILENAME} !-d
    RewriteRule ^(.*)$ index.php?url=$1 [QSA,L]
</IfModule>
```

2. Buat `public/index.php`:

```
<?php
require_once __DIR__ . '/../app/config/config.php';
require_once __DIR__ . '/../app/core/helpers.php';

spl_autoload_register(function ($class) {
    foreach (['core', 'controllers', 'models'] as $dir) {
        $file = ROOT_PATH . '/app/' . $dir . '/' . $class . '.php';
        if (file_exists($file)) { require_once $file; return; }
    }
});

// Fallback jika rewrite .htaccess tidak tersedia (PHP built-in server),
// sekaligus menangani subfolder, mis. http://localhost/inventaris-mvc/public/.
if (!isset($_GET['url'])) {
    $uri = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH);
```

```
    $basePath = str_replace('\\', '/', dirname($_SERVER['SCRIPT_NAME'] ?? ''));
    if ($basePath !== '/' && $basePath !== '' && str_starts_with($uri, $basePath)) {
        $uri = substr($uri, strlen($basePath));
    }
    $_GET['url'] = trim((string) $uri, '/');
}

session_start();
$app = new App();
```

3. Sementara buat `app/core/helpers.php` minimal (akan dilengkapi di Langkah 13):

```
<?php
function e($value): string {
    return htmlspecialchars((string) $value, ENT_QUOTES, 'UTF-8');
}
```

> **Catatan:** jangan jalankan dulu sebelum `App` dibuat (Langkah 3) — akan error karena class `App` belum ada.

Checkpoint: `php -l public/index.php` tidak menemukan error sintaks.

Langkah 3 — Router & Base Controller (Hello MVC)

Tujuan: memetakan URL `/{controller}/{method}/{param}` ke Controller, lalu menampilkan View.

Konsep: URL `/items/edit/3` dipecah menjadi `ItemsController` → `edit(3)`. Base `Controller` menyediakan `view()`, `model()`, `redirect()`, dan `flash()`.

Praktik

1. Buat `app/core/Controller.php`:

```
<?php
class Controller {
    public function view(string $view, array $data = []): void {
        extract($data);
        require ROOT_PATH . '/app/views/' . $view . '.php';
    }
    public function model(string $model) { return new $model(); }
    public function redirect(string $path): void {
        header('Location: ' . BASEURL . '/' . ltrim($path, '/'));
        exit;
    }
    public function flash(string $type, string $message): void {
        $_SESSION['flash'] = ['type' => $type, 'message' => $message];
    }
}
```

2. Buat `app/core/App.php` (router):

```
<?php
class App {
    protected $controller = 'HomeController';
    protected string $method = 'index';
    protected array $params = [];
```

```
    public function __construct() {
        $url = $this->parseURL();

        if (!empty($url[0])) {
            $candidate = ucfirst($url[0]) . 'Controller';
            if (file_exists(ROOT_PATH . '/app/controllers/' . $candidate . '.php')) {
                $this->controller = $candidate;
                unset($url[0]);
            } else { $this->notFound(); return; }
        }
        $this->controller = new $this->controller();

        if (!empty($url[1])) {
            if (method_exists($this->controller, $url[1])) {
                $this->method = $url[1]; unset($url[1]);
            } else { $this->notFound(); return; }
        }
        $this->params = $url ? array_values($url) : [];
        call_user_func_array([$this->controller, $this->method], $this->params);
    }

    public function parseURL(): array {
        if (isset($_GET['url'])) {
            $url = rtrim($_GET['url'], '/');
            return explode('/', filter_var($url, FILTER_SANITIZE_URL));
        }
        return [];
    }

    private function notFound(): void {
        http_response_code(404);
        echo '<h1>404 Not Found</h1>';
    }
}
```

> Catatan: `$controller` sengaja **tanpa** tipe `string` karena setelah `new` isinya objek. Kesalahan memberi tipe `string` adalah *bug* umum (`Cannot assign HomeController to property ... of type string`).

3. Buat `HomeController` dan view pertamanya `app/views/layouts/header.php` + `footer.php` sederhana, lalu `app/views/dashboard/index.php`:

```
<?php
class HomeController extends Controller {
    public function index(): void {
        $this->view('dashboard/index', ['title' => 'Dashboard']);
    }
}
```

4. Jalankan:

```
php -S localhost:8000 -t public public/index.php
```

   Buka `http://localhost:8000`.

Checkpoint: halaman "Dashboard" tampil. Membuka `/tidak-ada` memberi **404**.

Kesalahan umum: nama controller harus **Persis** `ucfirst(segmen) . 'Controller'` (mis. URL `items` → `ItemsController`, bukan `ItemController`).

Langkah 4 — Koneksi Database (PDO) & Base Model

Tujuan: menyediakan koneksi PDO tunggal (*singleton*) dan query dasar.

Konsep: `Database` mengembalikan satu objek `PDO`; `Model` memakai `$this->db` untuk query. `PDO::ATTR_ERRMODE => EXCEPTION` membuat error SQL melempar *exception*.

Praktik

1. `app/core/Database.php`:

```php
<?php
class Database {
    private static ?Database $instance = null;
    private PDO $pdo;

    private function __construct() {
        $dsn = 'mysql:host=' . DB_HOST . ';port=' . DB_PORT
             . ';dbname=' . DB_NAME . ';charset=utf8mb4';
        try {
            $this->pdo = new PDO($dsn, DB_USER, DB_PASS);
            $this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
            $this->pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
        } catch (PDOException $e) {
            die('Koneksi database gagal: ' . $e->getMessage());
        }
    }
    public static function getInstance(): Database {
        return self::$instance ??= new self();
    }
    public function pdo(): PDO { return $this->pdo; }
}
```

2. `app/core/Model.php`:

```php
<?php
class Model {
    protected PDO $db;
    protected string $table = '';

    public function __construct() {
        $this->db = Database::getInstance()->pdo();
    }
    public function all(): array {
        return $this->db->query("SELECT * FROM {$this->table}")->fetchAll();
    }
    public function find(int $id) {
        $stmt = $this->db->prepare("SELECT * FROM {$this->table} WHERE id = ?");
        $stmt->execute([$id]);
        return $stmt->fetch();
    }
    public function delete(int $id): bool {
        return $this->db->prepare("DELETE FROM {$this->table} WHERE id = ?")->execute([$id]);
    }
    public function count(): int {
        return (int) $this->db->query("SELECT COUNT(*) FROM {$this->table}")->fetchColumn();
    }
}
```

Checkpoint: buat berkas uji sementara:

```
php -r "require 'app/config/config.php'; require 'app/core/Database.php'; var_dump(Database::getInstance()->pdo() instanceof PDO);"
```

Setelah database dibuat (Langkah 5) koneksi akan benar-benar teruji.

Kesalahan umum: `charset=utf8` (lama) sebaiknya `utf8mb4`; menyimpan kredensial di kode, bukan `.env`.

Langkah 5 — Skema Database & Data Contoh

Tujuan: membuat database `db_inventory` dengan empat tabel dan data contoh.

Konsep: relasi **one-to-many** (*category* → *items*) dan relasi **transactions → items/users**. Gunakan *foreign key* dengan `ON DELETE SET NULL` / `CASCADE` yang tepat.

Praktik

1. Buat `database/inventory.sql` (ringkas):

```sql
CREATE DATABASE IF NOT EXISTS db_inventory
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE db_inventory;

DROP TABLE IF EXISTS transactions, items, categories, users;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  full_name VARCHAR(100),
  role ENUM('admin','staff','manager') NOT NULL DEFAULT 'staff',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  code VARCHAR(20) NOT NULL UNIQUE,
  name VARCHAR(150) NOT NULL,
  category_id INT NULL,
  quantity INT NOT NULL DEFAULT 0,
  unit VARCHAR(20) NOT NULL DEFAULT 'unit',
  price DECIMAL(15,2) NOT NULL DEFAULT 0,
  entry_date DATE NULL,
  description TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_items_category FOREIGN KEY (category_id)
    REFERENCES categories(id) ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE transactions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  item_id INT NOT NULL,
  user_id INT NOT NULL,
  type ENUM('IN','OUT') NOT NULL,
  quantity INT NOT NULL,
  notes VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_transactions_item FOREIGN KEY (item_id)
```

```
    REFERENCES items(id) ON DELETE CASCADE,
  CONSTRAINT fk_transactions_user FOREIGN KEY (user_id)
    REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

2. Tambahkan `INSERT` data contoh: 3 user, 4 kategori, 12 barang, 4 transaksi (lihat berkas `../inventory-mvc/database/inventory.sql`). Hash password untuk semua user `123456` dibuat dengan `password_hash('123456', PASSWORD_DEFAULT)`.
3. Impor:

   ```
   mysql -u root -p < database/inventory.sql
   ```

   atau via phpMyAdmin (*Import*).

> **Windows (XAMPP):** - Cara termudah: buka http://localhost/phpmyadmin → menu **Import** → pilih `database/inventory.sql`. - Command Prompt: `C:\xampp\mysql\bin\mysql -u root -p < database\inventory.sql` - PowerShell (tidak mendukung `<`): `Get-Content database\inventory.sql | C:\xampp\mysql\bin\mysql -u root -p`

Checkpoint:

```
mysql -u root -p -e "USE db_inventory; SHOW TABLES; SELECT COUNT(*) FROM items;"
```

Menampilkan 4 tabel dan 12 barang.

Kesalahan umum: urutan `DROP` salah (tabel anak harus dihapus lebih dulu); lupa `ON DELETE` sehingga hapus kategori gagal.

Langkah 6 — Layout View & Helper

Tujuan: membuat layout (header & footer) agar tampilan konsisten dan tidak berulang.

Konsep: view memakai `require` layout. Helper `e()` meng-*escape* output (anti-XSS) dan `rupiah()` memformat uang.

Praktik

1. Lengkapi `app/core/helpers.php`:

   ```
   function e($value): string {
       return htmlspecialchars((string) $value, ENT_QUOTES, 'UTF-8');
   }
   function rupiah($value): string {
       return 'Rp ' . number_format((float) $value, 0, ',', '.');
   }
   function base_url(string $path = ''): string {
       return BASEURL . '/' . ltrim($path, '/');
   }
   function take_flash(): ?array {
       if (empty($_SESSION['flash'])) return null;
       $f = $_SESSION['flash']; unset($_SESSION['flash']); return $f;
   }
   ```

2. `app/views/layouts/header.php` — HTML `<head>`, navbar, dan penampil *flash*:

```php
<?php $flash = take_flash(); ?>
<!DOCTYPE html>
<html lang="id"><head>
  <meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1">
  <title><?= isset($title) ? e($title) . ' - ' : '' ?><?= APP_NAME ?></title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head><body class="bg-light">
<main class="container py-4">
<?php if ($flash): ?>
  <div class="alert alert-<?= e($flash['type']) ?>"><?= e($flash['message']) ?></div>
<?php endif; ?>
```

3. `app/views/layouts/footer.php`:

```php
</main>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body></html>
```

4. View memakai layout: `require ROOT_PATH . '/app/views/layouts/header.php';` … konten … `require .../footer.php`.

Checkpoint: dashboard tampil dengan navbar Bootstrap; pesan *flash* muncul satu kali lalu hilang.

Langkah 7 — CRUD Kategori (Model → Controller → View)

Tujuan: melatih siklus lengkap MVC pada entitas sederhana (Kategori).

Konsep: alur CRUD = `index` (daftar), `create` (form), `store` (simpan), `edit` (form), `update`, `delete`.

Praktik

1. **Model** `app/models/Category.php`:

```php
<?php
class Category extends Model {
    protected string $table = 'categories';
    public function all(): array {
        return $this->db->query(
            "SELECT c.*, (SELECT COUNT(*) FROM items i WHERE i.category_id = c.id) AS item_count
             FROM categories c ORDER BY c.name"
        )->fetchAll();
    }
    public function create(array $data): bool {
        return $this->db->prepare(
            "INSERT INTO categories (name, description) VALUES (:name, :description)"
        )->execute($data);
    }
    public function update(int $id, array $data): bool {
        $data['id'] = $id;
        return $this->db->prepare(
            "UPDATE categories SET name = :name, description = :description WHERE id = :id"
        )->execute($data);
    }
}
```

2. **Controller** `app/controllers/CategoriesController.php` — `index`, `create`, `store`, `edit`, `update`, `delete`. Pada `store` / `update` validasi input lalu panggil model, set `flash('success',`

`...)`, dan `redirect('categories')`.

```
public function index(): void {
    $this->view('categories/index', [
        'title' => 'Data Kategori',
        'categories' => $this->model('Category')->all(),
    ]);
}
public function store(): void {
    if ($_SERVER['REQUEST_METHOD'] !== 'POST') $this->redirect('categories');
    $input = array_map('trim', $_POST);
    if (empty($input['name'])) {
        $this->flash('danger', 'Nama kategori wajib diisi.');
        $this->redirect('categories/create');
    }
    $this->model('Category')->create([
        'name' => $input['name'],
        'description' => $input['description'] ?? '',
    ]);
    $this->flash('success', 'Kategori berhasil ditambahkan.');
    $this->redirect('categories');
}
```

3. **View**: `categories/index.php` (tabel + tombol), `categories/create.php`, `categories/edit.php`, dan partial `categories/_form.php`.
4. **Hapus** memakai `<form method="post">` + konfirmasi JavaScript (bukan tautan GET) agar aman.

Checkpoint: tambah, ubah, hapus kategori dari browser. Data berubah di tabel `categories`.

Kesalahan umum: memakai tautan `GET` untuk hapus (sebaiknya POST); lupa `exit` setelah `header('Location: ...')`.

Langkah 8 — CRUD Barang

Tujuan: mengulang pola CRUD pada entitas yang lebih kaya (relasi ke kategori, format uang).

Praktik

1. **Model** `app/models/Item.php`: `all()` (JOIN kategori), `find()`, `create()`, `update()`, `delete()`, serta `generateCode()` otomatis (`ITM0001`, ...):

```
public function generateCode(): string {
    $max = $this->db->query(
        "SELECT MAX(CAST(SUBSTRING(code, 4) AS UNSIGNED)) FROM items WHERE code LIKE 'ITM%'"
    )->fetchColumn();
    return 'ITM' . str_pad((string)(((int)$max ? (int)$max : 0) + 1)), 4, '0', STR_PAD_LEFT);
}
```

2. **Controller** `app/controllers/ItemsController.php` — mirip Kategori, ditambah pengisian dropdown kategori pada form dan pemformatan nilai. Gunakan metode privat `payload()` untuk menormalkan input:

```
private function payload(array $input): array {
    return [
        'code'        => $input['code'],
        'name'        => $input['name'],
        'category_id' => ($input['category_id'] ?? '') !== '' ? (int)$input['category_id'] : null,
```

```
        'quantity'    => (int)($input['quantity'] ?? 0),
        'unit'        => $input['unit'] ?: 'unit',
        'price'       => (float)($input['price'] ?? 0),
        'entry_date'  => ($input['entry_date'] ?? '') !== '' ? $input['entry_date'] : null,
        'description' => $input['description'] ?? '',
    ];
}
```

3. **View** `items/index.php`, `items/create.php`, `items/edit.php`, `items/_form.php`. Tampilkan harga dengan `rupiah($row['price'])` dan beri tanda bila stok ≤ 5.

Checkpoint: CRUD barang berjalan; kolom harga tampil `Rp 12.500.000`.

Kesalahan umum: `category_id` kosong dikirim sebagai `''` → simpan sebagai `null`, bukan `0`, agar FK tidak error.

Langkah 9 — Transaksi Stok Masuk/Keluar

Tujuan: mencatat pergerakan stok dan **mengubah stok barang secara atomik**, menjaga stok tidak negatif.

Konsep: gunakan **database transaction** (`beginTransaction` / `commit` / `rollBack`). Bila setelah `OUT` stok menjadi negatif, **batalkan** seluruh operasi.

Praktik

1. **Model** `app/models/Transaction.php`:

```
public function create(array $data): bool {
    try {
        $this->db->beginTransaction();

        $this->db->prepare(
            "INSERT INTO transactions (item_id, user_id, type, quantity, notes)
             VALUES (:item_id, :user_id, :type, :quantity, :notes)"
        )->execute([
            ':item_id'  => $data['item_id'],
            ':user_id'  => $data['user_id'],
            ':type'     => $data['type'],       // 'IN' atau 'OUT'
            ':quantity' => $data['quantity'],
            ':notes'    => $data['notes'] ?? null,
        ]);

        $op = ($data['type'] === 'IN') ? '+' : '-';
        $this->db->prepare("UPDATE items SET quantity = quantity {$op} :quantity WHERE id = :item_id")
                 ->execute([':quantity' => $data['quantity'], ':item_id' => $data['item_id']]);

        $check = $this->db->prepare("SELECT quantity FROM items WHERE id = :item_id");
        $check->execute([':item_id' => $data['item_id']]);
        if ((int) $check->fetchColumn() < 0) { $this->db->rollBack(); return false; }

        $this->db->commit();
        return true;
    } catch (PDOException $e) {
        if ($this->db->inTransaction()) $this->db->rollBack();
        return false;
    }
}
```

2. **Controller** `app/controllers/TransactionsController.php` — `index` (riwayat) serta `stockIn` / `stockOut`:

```
public function stockIn(): void  { $this->store('IN',  'Barang masuk berhasil dicatat!'); }
public function stockOut(): void { $this->store('OUT', 'Barang keluar berhasil dicatat!'); }

private function store(string $type, string $msg): void {
    if ($_SERVER['REQUEST_METHOD'] !== 'POST') $this->redirect('transactions');
    $itemId = (int)($_POST['item_id'] ?? 0);
    $qty    = (int)($_POST['quantity'] ?? 0);
    if ($itemId <= 0 || $qty <= 0) { $this->flash('danger', 'Data tidak valid.'); $this->redirect('items'); }

    $ok = $this->model('Transaction')->create([
        'item_id' => $itemId, 'user_id' => $_SESSION['user']['id'],
        'type' => $type, 'quantity' => $qty, 'notes' => trim($_POST['notes'] ?? '') ?: null,
    ]);
    $this->flash($ok ? 'success' : 'danger',
        $ok ? $msg : 'Gagal. Stok tidak mencukupi.');
    $this->redirect('items');
}
```

3. **View**: tombol "Barang Masuk/Keluar" pada `items/index.php` membuka **modal Bootstrap**, dan halaman `transactions/index.php` menampilkan riwayat.

Checkpoint: barang masuk +10 → stok naik; barang keluar 9999 (melebihi stok) → **ditolak** dan stok tidak berubah, riwayat tidak bertambah.

Kesalahan umum: lupa `rollBack()` → riwayat transaksi tetap tercatat walau gagal; tidak memakai DB transaction → data tidak konsisten.

Langkah 10 — Autentikasi & Proteksi Route

Tujuan: hanya pengguna yang login dapat mengakses aplikasi.

Konsep: simpan data user di `$_SESSION['user']`; **middleware** sederhana memaksa login dan mengarahkan ke halaman login.

Praktik

1. **Model** `app/models/User.php`:

```
public function login(string $username, string $password) {
    $stmt = $this->db->prepare("SELECT * FROM users WHERE username = ? LIMIT 1");
    $stmt->execute([$username]);
    $user = $stmt->fetch();
    if ($user && password_verify($password, $user['password'])) {
        unset($user['password']);
        return $user;
    }
    return false;
}
```

2. **Middleware** `app/core/Middleware.php`:

```
class Middleware {
    public static function isLoggedIn(): bool {
        if (session_status() === PHP_SESSION_NONE) session_start();
        return !empty($_SESSION['user']);
    }
```

```
    public static function requireLogin(): void {
        if (!self::isLoggedIn()) {
            $_SESSION['flash'] = ['type' => 'danger', 'message' => 'Silakan login terlebih dahulu.'];
            header('Location: ' . BASEURL . '/auth/login'); exit;
        }
    }
    public static function user(): ?array { return $_SESSION['user'] ?? null; }
}
```

3. Tambahkan pada `Controller`:

```
protected function requireLogin(): void { Middleware::requireLogin(); }
```

   Lalu panggil `$this->requireLogin();` pada konstruktor controller yang dilindungi (`Home/Dashboard`, `Items`, `Categories`, `Transactions`, `Users`).

4. **AuthController** `app/controllers/AuthController.php` — `login` (form), `authenticate` (proses), `logout`:

```
public function authenticate(): void {
    if ($_SERVER['REQUEST_METHOD'] !== 'POST') $this->redirect('auth/login');
    $username = trim($_POST['username'] ?? '');
    $password = $_POST['password'] ?? '';
    $user = $this->model('User')->login($username, $password);
    if ($user) {
        $_SESSION['user'] = $user;
        $this->flash('success', 'Selamat datang, ' . $user['full_name'] . '!');
        $this->redirect('dashboard');
    }
    $this->flash('danger', 'Username atau password salah.');
    $this->redirect('auth/login');
}
```

5. **View** `app/views/auth/login.php` (form POST ke `auth/authenticate`), dan navbar menampilkan nama user + tombol logout.

Checkpoint: akses `/items` tanpa login → dialihkan ke `/auth/login`; login `admin` / `123456` → masuk dashboard; logout → keluar.

Kesalahan umum: menyimpan password **plaintext** (wajib `password_hash` / `password_verify`); membuat sesi setelah output (header error).

Langkah 11 — Paginasi, Pencarian, & Filter

Tujuan: menampilkan data dalam halaman-halaman dengan pencarian & filter kategori.

Konsep: `LIMIT` + `OFFSET`; hitung `total_pages` dari `COUNT(*)`. Selalu batasi nilai `limit` ke daftar putih.

Praktik

1. Tambahkan ke `Item`:

```
public function paginate(int $limit=10, int $page=1, string $search='', string $categoryId=''): array {
    $offset = ($page - 1) * $limit;
    $where = []; $params = [];
    if ($search !== '') {
        $where[] = "(i.code LIKE :search OR i.name LIKE :search OR c.name LIKE :search)";
        $params['search'] = '%' . $search . '%';
```

```
    }
    if ($categoryId !== '') {
        $where[] = "i.category_id = :category_id";
        $params['category_id'] = (int) $categoryId;
    }
    $whereSql = $where ? 'WHERE ' . implode(' AND ', $where) : '';

    $count = $this->db->prepare("SELECT COUNT(*) FROM items i
        LEFT JOIN categories c ON i.category_id = c.id {$whereSql}");
    $count->execute($params);
    $total = (int) $count->fetchColumn();

    $stmt = $this->db->prepare("SELECT i.*, c.name AS category_name
        FROM items i LEFT JOIN categories c ON i.category_id = c.id
        {$whereSql} ORDER BY i.id DESC LIMIT {$limit} OFFSET {$offset}");
    $stmt->execute($params);

    return ['data' => $stmt->fetchAll(), 'current_page' => $page,
            'total_pages' => (int) ceil($total / max($limit,1)), 'total_data' => $total];
}
```

2. **Controller** membaca `$_GET['search']`, `$_GET['category']`, `$_GET['page']`, `$_GET['limit']`, dengan *whitelist* limit `[5,10,20,50,100]`.
3. **View** menampilkan form pencarian, dropdown kategori, dan navigasi halaman (`Previous` / `1 2 3` / `Next`) memakai `http_build_query()`.

Checkpoint: `/items?search=laptop` menyaring hasil; `/items?limit=5&page=2` pindah halaman.

Kesalahan umum: interpolasi `search` langsung ke SQL (rawan injeksi) — gunakan *prepared statement*; `limit` tidak divalidasi.

Langkah 12 — Dashboard Dinamis

Tujuan: merangkum kondisi inventaris.

Praktik

1. Tambahkan metode ke `Item`: `totalItems()`, `totalStock()`, `totalAssetValue()`, `lowStockCount()`, `lowStockItems()`. Gunakan agregasi SQL:

```
public function totalAssetValue(): float {
    return (float) $this->db->query("SELECT COALESCE(SUM(quantity * price), 0) FROM items")-
>fetchColumn();
}
```

2. Tambahkan ke `Transaction`: `recent(int $limit)` dan `todayCount()` (`WHERE DATE(created_at) = CURDATE()`).
3. `HomeController` mengirim semua statistik ke view `dashboard/index.php`; tampilkan kartu statistik, tabel **stok menipis**, dan **transaksi terbaru**.

Checkpoint: kartu "Nilai Aset" menampilkan total rupiah; daftar transaksi terbaru mengikuti data.

Langkah 13 — Halaman 404, Logging, & Perapian Helper

Tujuan: menangani rute tak dikenal dengan rapi dan mencatat error.

Praktik

1. `app/core/Logger.php` menulis error ke `logs/errors/error-YYYYmmddHHMMSS.json`:

```
class Logger {
    private static string $logDir = ROOT_PATH . '/logs/errors';
    public static function error(string $message, array $context = []): void {
        if (!is_dir(self::$logDir)) mkdir(self::$logDir, 0777, true);
        file_put_contents(
            self::$logDir . '/error-' . date('YmdHis') . '.json',
            json_encode(['timestamp' => date('Y-m-d H:i:s'), 'message' => $message, 'context' => $context],
JSON_PRETTY_PRINT)
        );
    }
}
```

2. Gunakan `Logger::error()` di `Database` (gagal koneksi) dan `Transaction` (gagal commit).
3. Lengkapi `App::notFound()` agar me-render `app/views/errors/404.php` (dengan layout).
4. Pindahkan seluruh fungsi pembantu (`e`, `rupiah`, `base_url`, `take_flash`) ke `app/core/helpers.php` — **bukan** di `config.php` — agar `config.php` murni konfigurasi.

Checkpoint: `/halaman-ngawur` menampilkan halaman 404 berlayout; `logs/` dibuat saat terjadi error DB.

Langkah 14 — Pengujian Akhir & Checklist Serah Terima

Lakukan pengujian fungsional berikut (uji manual + tandai):

No	Skenario	Hasil Diharapkan
1	Buka `/` tanpa login	Dialihkan ke `/auth/login`
2	Login salah	Tetap di login, muncul pesan error
3	Login `admin` / `123456`	Masuk dashboard
4	Tambah / edit / hapus kategori	Data berubah, muncul *flash*
5	Tambah / edit / hapus barang	Data berubah, harga terformat
6	Barang masuk +10	Stok naik, riwayat bertambah
7	Barang keluar melebihi stok	Ditolak, stok **tidak** berubah, riwayat **tidak** bertambah
8	Pencarian & filter kategori	Hasil tersaring
9	Paginasi (`limit=5`)	Navigasi halaman benar
10	URL tidak dikenal	Halaman 404 berlayout
11	Logout	Session dihapus, kembali ke login

Git final:

```
git add .
git commit -m "feat: aplikasi inventaris MVC selesai"
git branch -M main
git remote add origin <url-repo-anda>
git push -u origin main
```

Buat `README.md` berisi cara instalasi, impor database, dan kredensial demo.

3. Isu Terkini & Praktik Industri

- **Framework MVC** (Laravel, CodeIgniter, Symfony) memakai konsep yang sama: router, controller, model, view, middleware, service container. Menguasai MVC "murni" memudahkan migrasi ke framework.
- **Keamanan:** selalu gunakan **prepared statement** (cegah SQL injection), **escape output** (cegah XSS), **password hashing** (`password_hash`), dan **CSRF token** untuk form (pengembangan lanjutan).
- **Layered architecture:** pemisahan Model–View–Controller dapat diperluas dengan **Service layer** dan **Repository** untuk aplikasi besar.
- **API/Headless:** aplikasi web modern sering mengekspos **REST API/JSON** sehingga front-end terpisah (SPA/mobile) dapat mengonsumsi data yang sama.

BAGIAN 3: RANGKUMAN & EVALUASI PEMBELAJARAN

1. Rangkuman

1. **MVC** memisahkan data (**Model**), tampilan (**View**), dan alur (**Controller**); *front controller* + *router* menjadi pintu masuk tunggal.
2. **Autoloader** (`spl_autoload_register`) memuat class otomatis; **PDO singleton** menyediakan koneksi tunggal.
3. **CRUD** mengikuti pola konsisten: `index – create – store – edit – update – delete`.
4. **Database transaction** menjamin konsistensi saat mengubah lebih dari satu tabel (stok + riwayat) dan mencegah stok negatif.
5. **Session + middleware** membatasi akses; password disimpan **ter-hash**.
6. **Paginasi/pencarian/filter** memakai `LIMIT/OFFSET` dan *prepared statement* dengan *whitelist* `limit`.
7. `.env` memusatkan konfigurasi; **helper** (`e`, `rupiah`) dan **Logger** menjaga kualitas dan keamanan kode.

2. Latihan Soal & Tugas Mandiri

A. Soal Analisis / Pilihan Ganda (HOTS)

1. Mengapa `$controller` pada `App.php` tidak boleh diberi tipe `string`? Jelaskan kaitannya dengan `new`.
2. Apa yang terjadi jika `Transaction::create()` tidak memakai `beginTransaction()` saat stok tidak mencukupi?
3. Jelaskan mengapa output wajib dilewatkan `e()` (htmlspecialchars) sebelum ditampilkan.
4. Bandingkan query `search` yang aman (prepared) vs yang rentan injeksi; tunjukkan contoh serangannya.
5. Kapan `ON DELETE SET NULL` lebih tepat daripada `ON DELETE CASCADE`?

B. Soal Esai Kasus

1. Gambarkan alur *request* `/items/edit/3` dari browser hingga HTML dihasilkan, sebutkan berkas dan method yang dilewati.
2. Rancang skema paginasi untuk 1.000 data barang dengan `limit` 20; tuliskan query `COUNT` dan query data.
3. Uraikan langkah menambahkan proteksi agar hanya `role = admin` yang boleh menghapus barang.

C. Tugas Praktik Berbasis CBL (Case-Based Learning)

Pilih **minimal dua** fitur tambahan berikut, kerjakan di repositori yang sama, dan commit secara bertahap:

1. **Export CSV** daftar barang (bonus nilai).
2. **Soft delete** kategori/barang (tandai `deleted_at` alih-alih hapus permanen).
3. **Role-based access control (RBAC)** sederhana: `staff` hanya boleh transaksi, `admin` boleh kelola master data.
4. **Detail barang** menampilkan riwayat transaksi per barang.
5. **Laporan stok menipis** dengan filter rentang tanggal.

BAGIAN 4: DAFTAR PUSTAKA & GLOSARIUM

1. Glosarium

Istilah	Arti
MVC	Pola arsitektur yang memisahkan Model, View, dan Controller.
Front Controller	Satu berkas pintu masuk yang menangani semua request.
Router	Pemeta URL ke controller/method.
Autoloader	Mekanisme memuat class otomatis berdasarkan nama.
PDO	PHP Data Objects — antarmuka akses database yang aman.
Prepared Statement	Query berparameter yang memisahkan SQL dari data (anti-injeksi).
Middleware	Lapisan pemeriksaan sebelum request diteruskan (mis. cek login).
Session	Penyimpanan data pengguna antar-request di server.
Transaction (DB)	Sekumpulan operasi yang harus berhasil/gagal bersama-sama.
Paginasi	Pemecahan data menjadi beberapa halaman.
XSS	Serangan menyisipkan skrip melalui output yang tidak di-escape.
SQL Injection	Serangan menyisipkan SQL melalui input yang tidak divalidasi.

2. Daftar Pustaka

1. Nixon, R. *Learning PHP, MySQL & JavaScript*. O'Reilly Media.
2. Sklar, D. *Learning PHP*. O'Reilly Media.
3. Chacon, S. & Straub, B. *Pro Git*. Apress.
4. Dokumentasi resmi PHP: https://www.php.net/manual/.
5. Dokumentasi resmi MySQL: https://dev.mysql.com/doc/.
6. PHP Standards Recommendations (PSR-12): https://www.php-fig.org/psr/psr-12/.

> **Referensi Kode Lengkap:** seluruh kode yang dibangun pada modul ini tersedia di folder `../inventory-mvc`. Gunakan untuk membandingkan hasil kerja Anda — **setelah** mencoba sendiri.